Bandit LAB - presentation script (read aloud)

Social Attention for Autonomous Decision-Making in Dense Traffic (Leurent & Mercat, NeurIPS 2019 workshop). 18 slides, about 10 minutes. Each speaker clicks through their own slides. Appendix slides A1-A5 are only for questions.

Planned speaking time: 9 min 15 s.

Speaker 1 - slides 1-5: problem and reinforcement learning (DQN) (~3 min 0 s)

Slide 1 (~25 s)

Good afternoon. We are Bandit LAB. Our project implements the paper “Social Attention for Autonomous Decision-Making in Dense Traffic” by Edouard Leurent and Jean Mercat, from INRIA and Renault, published at the NeurIPS 2019 workshop on machine learning for autonomous driving. We reproduced the paper, and then we extended it in two ways.

Slide 2 (~30 s)

The problem is an unprotected left turn. There is no traffic light and cars come from every direction. If our car goes too early, it crashes. If it waits too long, it never gets through. This is called the freezing robot problem. Hand-written rules do not cover every situation, so we let the car learn by trial and error. That is reinforcement learning.

Slide 3 (~40 s)

We write the task as a Markov Decision Process. The state is a table with one row per car: our car plus up to fourteen others. Each row has seven numbers: present or not, position x and y, velocity x and y, and the cosine and sine of the heading. Positions are divided by one hundred and velocities by twenty, so every number is between minus one and one. Once per second the agent chooses slower, idle or faster; steering is automatic. The reward is between zero and one per second depending on speed, minus five for a crash, and plus one for reaching the exit.

Slide 4 (~35 s)

The goal is to maximise the return: the sum of future rewards, where a reward k steps ahead is multiplied by gamma to the power k. We use gamma zero point nine five, so the agent looks about twenty seconds ahead. Q of s and a is the expected return after taking action a in state s. The best Q satisfies the Bellman optimality equation: value now equals reward plus gamma times the best value in the next state. Once we know Q, the policy is simply: pick the action with the highest Q.

Slide 5 (~50 s)

The state is continuous, so we cannot store Q in a table. DQN approximates Q with a neural network with weights theta. We train it towards the target y: reward plus gamma times the value of the next state, using the squared error as loss. Three tricks make this stable. One: a replay buffer of fifteen thousand transitions; we learn from random batches of sixty-four. Two: a target network, a frozen copy of the weights, updated every 512 steps. Three: Double DQN: the online network picks the next action and the target network scores it, which reduces over-estimation. Epsilon-greedy exploration starts at one and decays to zero point zero five. The code on the right is exactly these lines from our train dot py. Now my teammate will explain the paper's main idea.

Speaker 2 - slides 6-10: the paper's idea, attention maths and code (~2 min 45 s)

Slide 6 (~30 s)

The paper keeps DQN fixed and asks one question: how should the network read the other cars? A plain list has two problems: its size changes with the number of cars, and it depends on the order of the cars. But a good driving policy must be permutation invariant: if we only re-order the other cars, the decision must not change. That is the equation on this slide. Like the paper, we shuffle the order of the cars at every step.

Slide 7 (~30 s)

We compare three networks with about thirty thousand weights each, so the comparison is fair. FCN List flattens the table into one hundred and five numbers and uses two dense layers; it is not order invariant. CNN Grid draws a thirty-two by thirty-two bird's-eye grid with two-metre cells; it is order invariant but coarse. Ego-Attention is the paper's model. Our parameter counts match the paper's Table 1.

Slide 8 (~45 s)

This is the attention mechanism step by step. Each car's row goes through a small shared encoder and becomes a sixty-four number vector e. Our car, the ego car, makes a query q. Every car makes a key k and a value v. The score of car i is q dot k i, divided by the square root of the head size, thirty-two. A softmax turns the scores into weights alpha that add up to one; empty rows are masked out. The output is the weighted sum of the values. We use two heads, so this happens twice in parallel. The result is added to the ego encoding, and a decoder outputs the three Q-values.

Slide 9 (~30 s)

A small example. The scores are two for a fast car from the left, zero point five for a far car, zero for a car leaving, and minus one point five for ourselves. Softmax gives zero point seven two, zero point one six, zero point one and zero point zero two. So seventy-two percent of what the network uses comes from the dangerous car. Because the output is a weighted sum, re-ordering the cars does not change it, and it works for any number of cars.

Slide 10 (~30 s)

This is our attend function in common dot py. Each line matches an equation: encoders, the query from the ego car, keys and values from everyone, scaled dot product, the mask, softmax, weighted sum and the residual. A bonus is that we can see the attention: we draw the weights as lines from our car, as in the picture. Now our third member will present the experiments and results.

Speaker 3 - slides 11-18: experiments, results, extensions, demo, conclusion (~3 min 30 s)

Slide 11 (~25 s)

We implemented everything from scratch in PyTorch and copied the hyper-parameters from the authors' own code. Three agents, three seeds each, four thousand episodes per run: nine runs. The paper used one hundred and twenty seeds, so we show ninety-five percent confidence bands. For testing, every agent drives the same one hundred unseen scenarios with no exploration.

Slide 12 (~25 s)

Our training curves show the same three behaviours as the paper's Figure 4. FCN, in orange, is the fastest but crashes and has the shortest episodes: risky and blind. CNN, in green, is the slowest: too cautious. Ego-Attention, in blue, gets the highest reward: it waits when needed and uses gaps.

Slide 13 (~25 s)

On the test, attention succeeds in sixty-seven percent of scenarios with only twelve percent crashes. FCN crashes forty-one percent of the time, and CNN times out thirty-four percent of the time. With denser traffic than in training, the ranking stays the same: FCN then crashes sixty-one percent of the time.

Slide 14 (~35 s)

Then we went beyond the paper. Its Section 4 suggests giving the agent its destination. We did that: our car starts on any of four roads and goes left, straight or right: twelve routes. Each car gets the direction to its destination; for our car it is the goal, for other cars it acts like a turn signal. One attention agent learned all twelve routes and halved the crash rate compared with FCN, eighteen versus thirty-four percent, with the biggest gain on the left turn.

Slide 15 (~40 s)

Our second extension is a long journey through a city of six intersections to a finish line. A route planner picks the turns with a shortest-path search, and our attention agent decides when to go. At each junction we give it positions relative to that junction, so its skill transfers. Without any training in the city, attention

finished fifty-two and a half percent of journeys, FCN only thirty-four. Risks add up: eighty-five percent per junction becomes about fifty-seven percent per journey. Fine-tuning on the city made it worse, because it became more aggressive. We report that honestly.

Slide 16 (~30 s)

Here is our demo. The green band is the planned route. The lines show attention: thicker means more attention, and the two colours are the two heads. On the right are the Q-values and the chosen action. The camera follows the car to the finish line.

Slide 17 (~25 s)

To conclude: how the network reads the state matters as much as the learning algorithm. Attention is order-free, works for any number of cars, and shows what the agent is looking at. We reproduced the paper and extended it to twelve routes and a whole city. Limitations: it is a simulator, other drivers follow simple rules, and our car only controls its speed.

Slide 18 (~5 s)

Thank you. We are happy to take your questions.

If the TA asks (short answers)

Why DQN and not PPO? The paper uses DQN, and our actions are discrete (3 choices) - DQN's natural setting.

What is Double DQN? The online network chooses the next action and the target network scores it. Plain DQN uses one network for both and over-estimates values.

Why a replay buffer and a target network? Replay breaks the correlation between consecutive steps; the target network keeps the target steady for 512 steps.

Why gamma 0.95? Episodes are short (13 decisions); 0.95 looks about 1/(1-0.95) = 20 steps ahead.

What does permutation invariant mean? Re-ordering the other cars does not change the decision. Attention is a weighted sum, so the order cannot matter.

Why divide by the square root of d_h? Dot products of long random vectors grow with their length; dividing by sqrt(32) keeps the softmax from becoming too sharp.

Why only 3 seeds? Each run takes about an hour on a laptop; the paper used 120 seeds on a cluster. We show 95% confidence bands.

How does one agent drive a whole city? A route planner picks the turns; at each junction the agent sees positions relative to that junction, like in training.

Why did fine-tuning make it worse? Every second at full speed earns reward on a 90 s journey, so a short fine-tune made it more aggressive and it forgot caution.

Why can't it avoid obstacles? It only controls speed and roads have one lane per direction; that needs lane changes - future work.